1 EXPERIMENT

This study explores how developers interact with AI-powered coding assistants in industry inspired programming scenarios. By observing and comparing how participants approach manual coding tasks and tasks using an AI enabled coding assistant, we aim to understand how such tools influence coding practices, problem-solving approaches, and overall developer productivity and sustainability of both the produced software and development process.

2 METHODOLOGY

This study is designed as a full-day observational experiment to explore how developers interact with AI-powered coding assistants, specifically the "Q Developer" tool, in a Python programming context. The study uses a mixed-methods approach, integrating both quantitative and qualitative data to provide a comprehensive understanding of developer behaviour and experience, along with exploring the sustainability of the process and developed software.

2.1 PARTICIPANTS

Participants will be recruited from three distinct groups:

- Developers from BT
- University students with relevant programming experience
- Developers from DiffBlue (subject to availability)

All participants must meet a predefined baseline of Python proficiency to ensure consistency in task execution and data quality. Each participant will work individually to eliminate group dynamics and focus on personal interaction with the AI assistant. An initial short training session of using AI coding assistants will be provided if necessary to ensure participants fully understand how to extract the benefit from the AI coding assistant.

Each participant will complete all tasks. Participants will complete some of their tasks without AI assistance and some will be completed with the AI coding assistant available.

2.2 PILOT STUDY

Prior to the main experiment, a pilot study will be conducted to refine the research design and ensure the robustness of the methodology. The pilot will involve a small number of participants of the GENIUS UK consortium and will be used to:

- **Validate the experimental setup**: This includes testing the technical environment (IDE, Git, AI plugin, CI/CD pipeline, virtual environment, and screen recording tools) to ensure smooth operation and usability.
- **Refine data collection instruments**: Insights from the pilot will inform the development of post-task surveys and semi-structured interview questions, ensuring they are relevant, clear, and capable of capturing meaningful data.

- **Identify usability and methodological issues**: The pilot will help uncover any practical or procedural challenges, such as unclear task instructions, technical glitches, or participant confusion, allowing for adjustments before the full study rollout.

Findings from the pilot will be documented and used to optimize the main study protocol, improving both the reliability and validity of the research outcomes.

2.3 Experimental Setup

To ensure consistency and control across all sessions, each participant will be provided with a fully standardized and isolated development environment tailored for Python development. This setup is designed to closely mirror industry-standard tooling and workflows, ensuring that participants engage with technologies and practices that reflect real-world development contexts. This environment will be pre-configured with all necessary tools and resources to support the use of the Q Developer AI assistant and facilitate smooth task execution.

The setup will include:

- **Python IDE**: A modern integrated development environment configured with relevant extensions to support Python and AI assistant integration. For the experiment Visual Studio Code will be used.
- **Access to a pre-existing codebase**: Participants will be provided with a realistic, appropriately complex given the timescales, codebase to work from. This will ensure tasks are grounded in practical development scenarios. This initial codebase will serve as the starting point for the assigned tasks and will include unit tests to support development and validation.
- **Existing documentation and requirements:** we will provide a set of requirements that forms the basis of the provided codebase.
- **Test Suites**: Three test suites will be provided. The first is a set of unit-tests that gives participants a baseline for correctness against the existing project. For the tasks, they should write the tests themselves. The second set of tests is not directly provided to participants and helps measure the performance of the solution to the tasks. A third set of tests will check for correctness.
- **Git version control**: Participants will have access to a GitLab based Git repository to manage code changes. The main branch will be protected and participants will create a feature branch. This will be isolated from other participants.
- **Q Developer AI plugin**: The AI assistant will be installed and configured to ensure consistent functionality across all sessions. All functionalities including in-line code completion, chat and agentic coding with be enabled and available to participants.
- **CI/CD pipeline configuration**: A basic continuous integration and deployment setup will be included to reflect industry development environments and allow participants to test and deploy code. As part of the CI/CD pipeline, a set of performance tests will also be included to measure the effectiveness of a participant's solution.

- **Python virtual environment (venv)**: Each participant will work within a clean virtual environment to manage dependencies and avoid conflicts.
- **Screen recording software**: All sessions will be recorded to capture participant interactions, coding behaviour, and use of the AI assistant.

Each participant's environment will be spun up independently to ensure isolation, reproducibility, and the ability to reset or reconfigure as needed. This approach minimizes variability and supports robust data collection and analysis.

Equipment for the experiment will be provided to participants, configured with the required software and environment as previously described.

2.4 PROCEDURE

2.4.1 TASK EXECUTION

Participants will be asked to complete a series of programming tasks, detailed below, using the Q Developer AI assistant. These tasks will be based on a specially crafted and representative codebase and designed to reflect realistic development scenarios, such as code optimisation, feature implementation, and architectural refactoring. Each task will be clearly defined with objectives and expected outcomes. Participants will work independently, and their interactions with the AI assistant will be unobtrusively recorded using screen capture and system logging tools. This will allow researchers to observe natural usage patterns without interfering with the development process.

2.4.2 Quantitative Data Collection

Quantitative data will be gathered from multiple sources to measure participant performance and interaction with the AI assistant. These metrics are essential for identifying patterns, assessing tool effectiveness, comparing participant outcomes, and evaluating sustainability impacts.

2.4.2.1 ***Usage metrics:*** *This includes the frequency and timing of AI assistant queries, the number of suggestions accepted or rejected, and the time spent on each task. These metrics help quantify engagement with the assistant and provide insight into how often and in what contexts developers rely on AI support. Usage metrics will be captured through plugin telemetry, IDE event logs, and timestamped screen recordings.*

2.4.2.2 ***Performance indicators:*** *These include task completion rates, the number of errors introduced or resolved, adherence to unit test requirements, and the performance of solutions to designated software optimisation tasks. For these optimisation tasks, performance will be measured using integration and performance tests embedded in the CI/CD pipeline. This allows for objective assessment of runtime efficiency, resource usage, and scalability of the submitted code. Collecting this data is crucial for evaluating the impact of AI assistance on code quality and developer productivity. Performance indicators will be assessed and collected through automated test results, Git commit analysis, CI/CD pipeline performance logs, and manual review of task outcomes.*

2.4.2.3 ***System-level data:*** *Git commits, CI/CD pipeline activity will be collected to provide a detailed view of development behaviour. This data helps contextualize participant actions and supports the identification of workflow patterns, bottlenecks, and deviations from expected practices. It also enables analysis of resource-intensive operations, helping to assess whether AI assistance leads to more sustainable development practices. System-level data will be extracted from integrated logging tools, Git repositories, and CI/CD dashboards.*

2.4.3 Qualitative Data Collection

Qualitative insights will be captured through multiple methods to explore the subjective experiences and contextual factors influencing developer interaction with the AI assistant. These insights will complement the quantitative findings and help interpret the broader implications of AI-assisted development, including sustainability.

2.4.3.1 ***Post-task surveys:*** *Participants will complete structured questionnaires after completing the tasks. These surveys will assess perceptions of the AI assistant's usability, trustworthiness, and impact on productivity amongst other things. Additional questions will explore whether participants felt the assistant helped them work more efficiently or sustainably (e.g., reducing time, effort, or unnecessary computation).*

2.4.3.2 ***Semi-structured interviews:*** *Conducted after initial data analysis, these interviews will delve deeper into participants' reasoning, expectations, and reflections. Interview prompts will be informed by themes emerging from observational and quantitative data, and will include questions about perceived benefits, limitations, and sustainability implications of using AI assistance. Interviews will be audio-recorded and transcribed for thematic analysis.*

2.5 Ethical Considerations

Ethical approval will be sought through King's College London (KCL) prior to participant recruitment.

3 High Level Scenario Description

The study scenario is situated within the context of a field engineer scheduling tool, which provides a realistic and complex domain for evaluating how developers interact with AI coding assistants. This scheduling problem is highly relevant to BT's industrial context, where efficient allocation of engineering resources across geographic regions and time constraints is a critical operational challenge. The tool includes scheduling logic that assigns jobs to engineers based on factors such as location, timing, and skill requirements. Experiment participants will engage with this codebase to complete tasks that reflect common software engineering challenges.

The tasks are designed to assess how AI assistance influences developer performance, decision-making, and code quality. They include:

- **Optimization tasks**: Participants will improve suboptimal implementations, such as a brute-force Traveling Salesman Problem (TSP) algorithm used to minimize engineer travel time, and a naïve bucket/bin sort algorithm used to match engineers to jobs.
- **Feature development**: Tasks include implementing a report generation feature that outputs assigned jobs per engineer, and replacing hardcoded job data with logic that reads from structured files.

- **Refactoring tasks**: Participants will decouple tightly coupled optimization routines to improve modularity and maintainability.

These tasks are timeboxed and can be completed manually or with assistance from an AI coding assistant. The scenario and task design aim to simulate real-world development conditions and provide insight into the practical utility and limitations of AI support in software engineering workflows.

For manual task completion, participants are welcome to use any sources of information, such as search engine results, stack overflow, etc, without using any AI assistance in the IDE. Users will be guided not to use AI chatbots such as ChatGPT.

For tasks where users are told they can use the AI assistant, they will have the freedom to complete any which way they can. If they wish to use AI assistances outside the IDE, that is also acceptable.

4 Tasks Descriptions for Participants

4.1 Optimization – Improve Engineer Routing

Objective: Refactor the current implementation of the engineer routing logic, which uses a brute-force approach to solve a Traveling Salesman Problem (TSP).

Instructions:

- Review the existing routing function used to minimize travel time between job locations.
- Identify inefficiencies in the current algorithm.
- Replace or improve the logic using a more efficient approach
- Ensure the updated implementation passes all existing unit tests and integrates cleanly with the scheduling workflow.

4.2 Optimization – Enhance Job Matching Algorithm

Objective: Improve the performance and accuracy of the job-to-engineer matching logic.

Instructions:

- Examine the current bucket/bin sort algorithm used to assign jobs based on location and skill.
- Refactor or redesign the algorithm to improve speed and matching precision.
- Consider edge cases such as overlapping skill sets or geographic constraints.
- Validate your changes using the provided test suite.

4.3 Feature Development – Job Assignment Report

Objective: Implement a feature that generates a report of job assignments per engineer.

Instructions:

- Create a function that outputs a structured summary of which jobs are assigned to which engineers.
- Format the report as a readable text or CSV file.
- Ensure the report includes relevant details such as job ID, location, time, and required skills.
- Include the travel time to a specified job.
- Integrate the feature into the existing codebase and confirm it runs correctly via unit tests.

4.4 Feature Development – External Job Data Integration

Objective: Replace hardcoded job data with logic that reads from structured external files.

Instructions:

- Modify the codebase to load job data from a provided JSON or CSV file.
- Ensure the data is parsed correctly and integrated into the scheduling logic.
- Update any affected functions or modules to accommodate dynamic data input.
- Confirm functionality with the provided test cases.

4.5 Refactoring – Decouple Optimization Logic

Objective: Improve code modularity by separating optimization routines from scheduling logic.

Instructions:

- Identify tightly coupled components in the scheduling and optimization modules.
- Refactor the code to isolate optimization logic into standalone functions or classes.
- Ensure the refactored code maintains existing functionality and passes all tests.
- Document your changes to support future maintainability.

5 Provided Codebase Creation

What are the factors we need to consider, especially from a sustainability perspective and what methodology should be followed?

6 Survey Questions & Interview Protocol

To be written